

It doesn't matter, of course, the order in which we put the names into the hash. Here is a piece of code to loop over years, printing the people born in that year:

```
string str;
for (Int i=1750;i<1800;i++) {
    if (person.getinit(i)) {
        cout << '\n' << "born in " << i << ":\n";
        while (person.getnext(str)) cout << str.data() << '\n';
    }
}
```

which gives as output

```
born in 1755:
Marie Antoinette

born in 1767:
Andrew Jackson
John Quincy Adams

born in 1770:
Ludwig van Beethoven

born in 1775:
Jane Austen

born in 1791:
Charles Babbage
James Buchanan
Samuel Morse
```

Notice that we could *not* have used null terminated C strings in this example, because C++ does not regard them as *first-class objects* that can be stored as elements of a vector. When you are using Hash or Mhash with strings, you will usually be better off using the C++ string class.

In §21.2 and §21.8 we will make extensive use of both the Hash and Mhash classes and almost all their member functions; look there for further usage examples.

By the way, Mr. Hacher's name is from the French *hacher*, meaning “to mince or hash.”

CITED REFERENCES AND FURTHER READING:

- Stroustrup, B. 1997, *The C++ Programming Language*, 3rd ed. (Reading, MA: Addison-Wesley), §17.6.2.[1]
- Knuth, D.E. 1997, *Sorting and Searching*, 3rd ed., vol. 3 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), §6.4–§6.5.
- Vitter, J.S., and Chen, W-C. 1987, *Design and Analysis of Coalesced Hashing* (New York: Oxford University Press).

7.7 Simple Monte Carlo Integration

Inspirations for numerical methods can spring from unlikely sources. “Splines” first were flexible strips of wood used by draftsmen. “Simulated annealing” (we shall

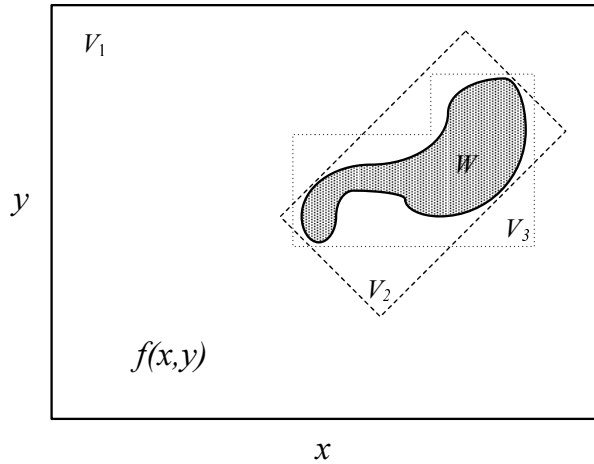


Figure 7.7.1. Monte Carlo integration of a function $f(x, y)$ in a region W . Random points are chosen within an area V that includes W and that can easily be sampled uniformly. Of the three possible V 's shown, V_1 is a poor choice because W occupies only a small fraction of its area, while V_2 and V_3 are better choices.

see in §10.12) is rooted in a thermodynamic analogy. And who does not feel at least a faint echo of glamor in the name “Monte Carlo method”?

Suppose that we pick N random points, uniformly distributed in a multidimensional volume V . Call them $x_0, \dots, x_{N-1}$. Then the basic theorem of Monte Carlo integration estimates the integral of a function f over the multidimensional volume,

$$\int f \, dV \approx V \langle f \rangle \pm V \sqrt{\frac{\langle f^2 \rangle - \langle f \rangle^2}{N}} \quad (7.7.1)$$

Here the angle brackets denote taking the arithmetic mean over the N sample points,

$$\langle f \rangle \equiv \frac{1}{N} \sum_{i=0}^{N-1} f(x_i) \quad \langle f^2 \rangle \equiv \frac{1}{N} \sum_{i=0}^{N-1} f^2(x_i) \quad (7.7.2)$$

The “plus-or-minus” term in (7.7.1) is a one standard deviation error estimate for the integral, not a rigorous bound; further, there is no guarantee that the error is distributed as a Gaussian, so the error term should be taken only as a rough indication of probable error.

Suppose that you want to integrate a function g over a region W that is not easy to sample randomly. For example, W might have a very complicated shape. No problem. Just find a region V that *includes* W and that *can* easily be sampled, and then define f to be equal to g for points in W and equal to zero for points outside of W (but still inside the sampled V). You want to try to make V enclose W as closely as possible, because the zero values of f will increase the error estimate term of (7.7.1). And well they should: Points chosen outside of W have no information content, so the effective value of N , the number of points, is reduced. The error estimate in (7.7.1) takes this into account.

Figure 7.7.1 shows three possible regions V that might be used to sample a complicated region W . The first, V_1 , is obviously a poor choice. A good choice, V_2 ,

can be sampled by picking a pair of uniform deviates (s, t) and then mapping them into (x, y) by a linear transformation. Another good choice, V_3 , can be sampled by, first, using a uniform deviate to choose between the left and right rectangular subregions (in proportion to their respective areas!) and, then, using two more deviates to pick a point inside the chosen rectangle.

Let's create an object that embodies the general scheme described. (We will discuss the implementing code later.) The general idea is to create an `MCIntegrate` object by providing (as constructor arguments) the following items:

- a vector `xlo` of lower limits of the coordinates for the rectangular box to be sampled
- a vector `xhi` of upper limits of the coordinates for the rectangular box to be sampled
- a vector-valued function `funcs` that returns as its components one or more functions that we want to integrate simultaneously
- a boolean function that returns whether a point is in the (possibly complicated) region W that we want to integrate; the point will already be within the region V defined by `xlo` and `xhi`
- a mapping function to be discussed below, or `NULL` if there is no mapping function or if your attention span is too short
- a seed for the random number generator

The object `MCIntegrate` has this structure.

```
struct MCIntegrate {
```

[mcintegrate.h](#)

Object for Monte Carlo integration of one or more functions in an `ndim`-dimensional region.

```
    Int ndim,nfun,n;           Number of dimensions, functions, and points sampled.
    VecDoub ff,fferr;          Answers: The integrals and their standard errors.
    VecDoub xlo,xhi,x,xx,fn,sf,sferr;
    Doub vol;                  Volume of the box  $V$ .
```

```
    VecDoub (*funcsp)(const VecDoub &);           Pointers to the user-supplied functions.
    VecDoub (*xmapp)(const VecDoub &);
    Bool (*inregionp)(const VecDoub &);
    Ran ran;                                       Random number generator.
```

```
    MCIntegrate(const VecDoub &xlow, const VecDoub &xhigh,
    VecDoub funcs(const VecDoub &), Bool inregion(const VecDoub &),
    VecDoub xmap(const VecDoub &), Int ranseed);
```

Constructor. The arguments are in the order described in the itemized list above.

```
void step(Int nstep);
Sample an additional nstep points, accumulating the various sums.
```

```
void calcanswers();
Calculate answers ff and fferr using the current sums.
```

```
};
```

The member function `step` adds sample points, the number of which is given by its argument. The member function `calcanswers` updates the vectors `ff` and `fferr`, which contain respectively the estimated Monte Carlo integrals of the functions and the errors on these estimates. You can examine these values, and then, if you want, call `step` and `calcanswers` again to further reduce the errors.

A worked example will show the underlying simplicity of the method. Suppose that we want to find the weight and the position of the center of mass of an object of

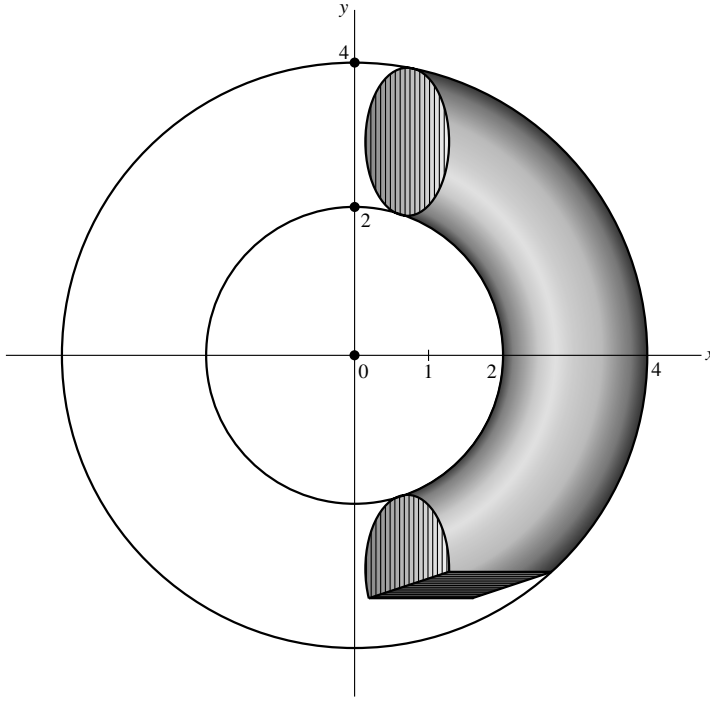


Figure 7.7.2. Example of Monte Carlo integration (see text). The region of interest is a piece of a torus, bounded by the intersection of two planes. The limits of integration of the region cannot easily be written in analytically closed form, so Monte Carlo is a useful technique.

complicated shape, namely the intersection of a torus with the faces of a large box. In particular, let the object be defined by the three simultaneous conditions:

$$z^2 + \left(\sqrt{x^2 + y^2} - 3 \right)^2 \leq 1 \quad (7.7.3)$$

(torus centered on the origin with major radius = 3, minor radius = 1)

$$x \geq 1 \quad y \geq -3 \quad (7.7.4)$$

(two faces of the box; see Figure 7.7.2). Suppose for the moment that the object has a constant density $\rho = 1$.

We want to estimate the following integrals over the interior of the complicated object:

$$\int \rho \, dx \, dy \, dz \quad \int x\rho \, dx \, dy \, dz \quad \int y\rho \, dx \, dy \, dz \quad \int z\rho \, dx \, dy \, dz \quad (7.7.5)$$

The coordinates of the center of mass will be the ratio of the latter three integrals (linear moments) to the first one (the weight).

To use the `MCintegrate` object, we first write functions that describe the integrands and the region of integration W inside the box V .

```
VecDoub torusfuncs(const VecDoub &x) {
```

```
mcintegrate.h
```

Return the integrands in equation (7.7.5), with $\rho = 1$.

```
    Doub den = 1.;
    VecDoub f(4);
    f[0] = den;
    for (Int i=1;i<4;i++) f[i] = x[i-1]*den;
    return f;
}
```

```
Bool torusregion(const VecDoub &x) {
```

Return the inequality (7.7.3).

```
    return SQR(x[2])+SQR(sqrt(SQR(x[0])+SQR(x[1]))-3.) <= 1.;
}
```

The code to actually do the integration is now quite simple,

```
VecDoub xlo(3), xhi(3);
xlo[0] = 1.; xhi[0] = 4.;
xlo[1] = -3.; xhi[1] = 4.;
xlo[2] = -1.; xhi[2] = 1.;
MCintegrate mymc(xlo,xhi,torusfuncs,torusregion,NULL,10201);
mymc.step(1000000);
mymc.calcanswers();
```

Here we've specified the box V by `xlo` and `xhi`, created an instance of `MCintegrate`, sampled a million times, and updated the answers `mymc.ff` and `mymc.fferr`, which can be accessed for printing or another use.

7.7.1 Change of Variables

A change of variable can often be extremely worthwhile in Monte Carlo integration. Suppose, for example, that we want to evaluate the same integrals, but for a piece of torus whose density is a strong function of z , in fact varying according to

$$\rho(x, y, z) = e^{5z} \quad (7.7.6)$$

One way to do this is, in `torusfuncs`, simply to replace the statement

```
Doub den = 1.;
```

by the statement

```
Doub den = exp(5.*x[2]);
```

This will work, but it is not the best way to proceed. Since (7.7.6) falls so rapidly to zero as z decreases (down to its lower limit -1), most sampled points contribute almost nothing to the sum of the weight or moments. These points are effectively wasted, almost as badly as those that fall outside of the region W . A change of variable, exactly as in the transformation methods of §7.3, solves this problem. Let

$$ds = e^{5z} dz \quad \text{so that} \quad s = \frac{1}{5}e^{5z}, \quad z = \frac{1}{5}\ln(5s) \quad (7.7.7)$$

Then $\rho dz = ds$, and the limits $-1 < z < 1$ become $.00135 < s < 29.682$.

The `MCintegrate` object knows that you might want to do this. If it sees an argument `xmap` that is not `NULL`, it will assume that the sampling region defined by `xlo` and `xhi` is not in physical space, but rather needs to be mapped into physical space before either the functions or the region boundary are calculated. Thus, to effect our change of variable, we *don't* need to modify `torusfuncs` or `torusregion`, but we *do* need to modify `xlo` and `xhi`, as well as supply the following function for the argument `xmap`:

mcintegrate.h

VecDoub torusmap(const VecDoub &s) {

Return the mapping from s to z defined by the last equation in (7.7.7), mapping the other coordinates by the identity map.

```
    VecDoub xx(s);
    xx[2] = 0.2*log(5.*s[2]);
    return xx;
}
```

Code for the actual integration now looks like this:

```
VecDoub slo(3), shi(3);
slo[0] = 1.; shi[0] = 4.;
slo[1] = -3.; shi[1] = 4.;
slo[2] = 0.2*exp(5.*(-1.)); shi[2] = 0.2*exp(5.*(1.));
MCIntegrate mymc2(slo,shi,torusfuncs,torusregion,torusmap,10201);
mymc2.step(1000000);
mymc2.calcanswers();
```

If you think for a minute, you will realize that equation (7.7.7) was useful only because the part of the integrand that we wanted to eliminate (e^{5z}) was both integrable analytically and had an integral that could be analytically inverted. (Compare §7.3.2.) In general these properties will not hold. Question: What then? Answer: Pull out of the integrand the “best” factor that *can* be integrated and inverted. The criterion for “best” is to try to reduce the remaining integrand to a function that is as close as possible to constant.

The limiting case is instructive: If you manage to make the integrand f *exactly* constant, and if the region V , of known volume, *exactly* encloses the desired region W , then the average of f that you compute will be exactly its constant value, and the error estimate in equation (7.7.1) will exactly vanish. You will, in fact, have done the integral exactly, and the Monte Carlo numerical evaluations are superfluous. So, backing off from the extreme limiting case, *to the extent* that you are able to make f approximately constant by change of variable, and *to the extent* that you can sample a region only slightly larger than W , you will increase the accuracy of the Monte Carlo integral. This technique is generically called *reduction of variance* in the literature.

The fundamental disadvantage of simple Monte Carlo integration is that its accuracy increases only as the square root of N , the number of sampled points. If your accuracy requirements are modest, or if your computer is large, then the technique is highly recommended as one of great generality. In §7.8 and §7.9 we will see that there are techniques available for “breaking the square root of N barrier” and achieving, at least in some cases, higher accuracy with fewer function evaluations.

There should be nothing surprising in the implementation of MCIntegrate. The constructor stores pointers to the user functions, makes an otherwise superfluous call to funcs just to find out the size of returned vector, and then sizes the sum and answer vectors accordingly. The step and calcanswer methods implement exactly equations (7.7.1) and (7.7.2).

mcintegrate.h

```
MCIntegrate::MCIntegrate(const VecDoub &xlow, const VecDoub &xhigh,
    VecDoub funcs(const VecDoub &), Bool inregion(const VecDoub &),
    VecDoub xmap(const VecDoub &), Int ranseed)
: ndim(xlow.size()), n(0), xlo(xlow), xhi(xhigh), x(ndim), xx(ndim),
  funcsp(funcs), xmap(xmap), inregionp(inregion), vol(1.), ran(ranseed) {
    if (xmap.nfun = funcs(xmap(xlo)).size();
    else nfun = funcs(xlo).size();
    ff.resize(nfun);
    fferr.resize(nfun);
```